Machine Learning

Lecture 11: Support Vector Machines — Margins, Duality, Kernels, and Solvers

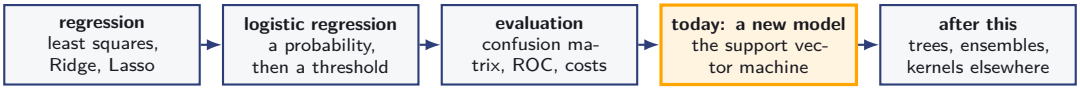
Instructor: Prof. Udit Bhatia

Civil Engineering; Computer Science and Engineering

Semester I, 2026



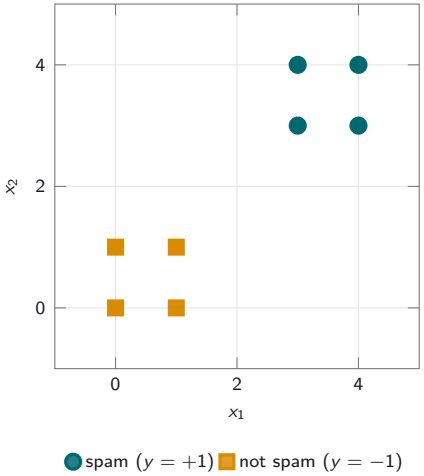
What This Lecture Builds, And Why It Is Built Slowly



So far every classifier was logistic regression, and we learned to judge any classifier. Today we build a second one, and it is worth building slowly, because its construction is a template for a whole way of thinking:

1. say in words what a *good* separating line is, then turn that sentence into an **optimisation problem**;
2. solve it, and discover that only a few data points matter (the **support vectors**);
3. notice the solution only touches the data through **inner products**;
4. replace those inner products by a **kernel**, and a straight line becomes any shape you like;
5. then ask what an **algorithm** for all this costs.

One Dataset For The Whole Lecture



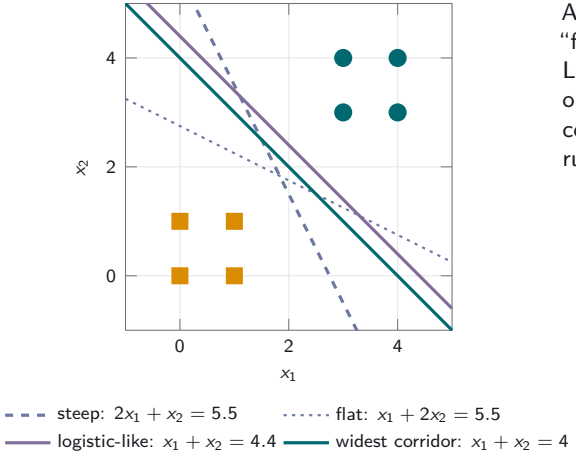
Eight emails, two features each: x_1 is the number of links and x_2 the number of words written in capitals. Both are small counts, so that every number in this lecture can be checked by hand. The label is $y_i \in \{+1, -1\}$ rather than $\{1, 0\}$.

i	1	2	3	4	5	6	7	8
x_1	3	4	3	4	1	0	1	0
x_2	3	3	4	4	1	1	0	0
y	$+1$	$+1$	$+1$	$+1$	-1	-1	-1	-1

Insight

Why ± 1: with a score $s(\mathbf{x})$ that is positive for spam and negative for other email, "point i is classified correctly" becomes the single inequality $y_i\, s(\mathbf{x}_i) > 0$. That trick is used on every slide from here on.

Many Lines Separate The Data. Which One Should We Pick?



All four lines classify all eight training points correctly, so "fewest training errors" cannot choose between them. Logistic regression picks one by maximum likelihood, but on separable data its likelihood keeps improving as the coefficients grow, so its line is decided by how long you run the optimiser, not by the geometry.

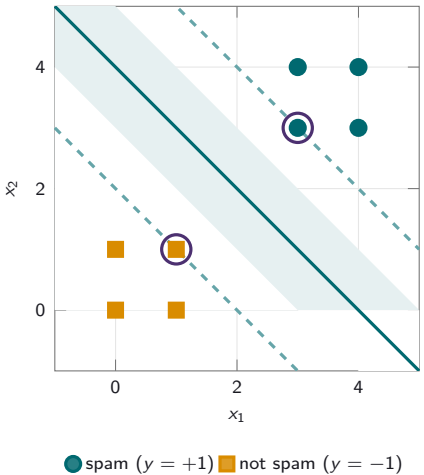
line	distance to its nearest point
steep	1.118
flat	1.118
logistic-like	1.131
$x_1 + x_2 = 4$	**1.414** (largest)

Question

A new email will land near the old ones. Which line leaves it the most room?

Interactive explorer: drag the points and watch which line the margin picks ↗

The Rule: Stay As Far As Possible From The Nearest Point



the widest empty corridor: no point inside the shaded band, two points (circled) touching its edges

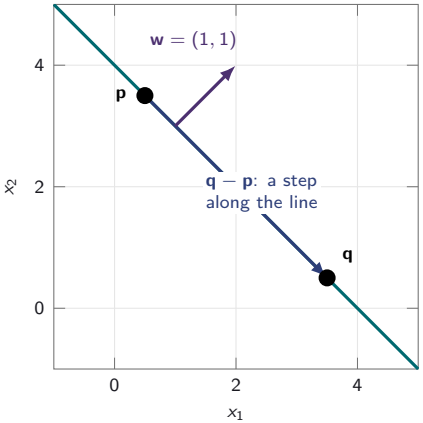
The philosophy. A line that grazes a training point is one small nudge away from an error. A line with a wide empty corridor on both sides tolerates noise, measurement error and new data. So among all lines that separate the classes, choose the one whose *nearest* point is *farthest* away.

What this needs. Three things must be made precise before it is an optimisation problem:

1. what "the distance from a point to a line" is, as a formula;
2. what "nearest" means when the points are on two sides;
3. what we are free to choose, and what is forced.

The next five slides do exactly that, in that order. The problem we end up with is the support vector machine.

A Line Is A Direction w And An Offset b; w Points Across The Line



the line $x_1 + x_2 - 4 = 0$ (teal), two points $\mathbf{p}, \mathbf{q}$ on it, and the direction $\mathbf{w}$ (purple), perpendicular to it

Write a line as the set of points $\mathbf{x}$ with

$$\mathbf{w}^\top \mathbf{x} + b = 0, \qquad \text{here } (1,1)\cdot\mathbf{x} - 4 = 0, \text{ i.e. } x_1 + x_2 = 4.$$

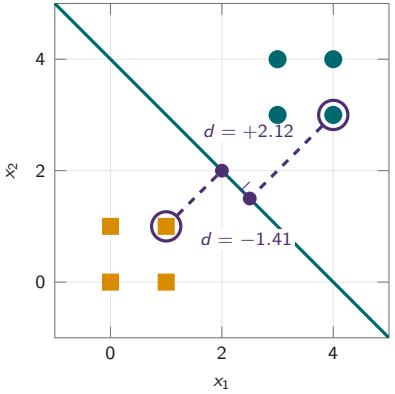
$\mathbf{w}$ is **not a point on the line**. It is a direction. To see which direction, take two points on the line, $\mathbf{p} = (0.5, 3.5)$ and $\mathbf{q} = (3.5, 0.5)$. The arrow from $\mathbf{p}$ to $\mathbf{q}$ is the vector $\mathbf{q} - \mathbf{p} = (3, -3)$: a *step along the line* (not a distance; it is a direction with a length). Both endpoints satisfy the line's equation, so subtracting the two equations,

$$\mathbf{w}^\top \mathbf{q} + b - (\mathbf{w}^\top \mathbf{p} + b) = \mathbf{w}^\top (\mathbf{q} - \mathbf{p}) = (1,1)\cdot(3,-3) = 0.$$

A zero dot product means perpendicular: $\mathbf{w}$ is at right angles to every step along the line. It points *across* the line, toward the side where $\mathbf{w}^\top \mathbf{x} + b > 0$.

The number $s(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ is the **score** of a point: zero on the line, positive on the side $\mathbf{w}$ points to, negative on the other side. Scores: $(3,3) \mapsto +2$, $(1,1) \mapsto -2$, $(2,2) \mapsto 0$.

How Far Is A Point From The Line? Walk Along w Until You Hit It



dashed: the perpendicular from each circled point to the line; its length is the distance. Small dots: the feet $(2.5, 1.5)$ and $(2, 2)$

Start at a point $\mathbf{x}$ and walk straight toward the line, i.e. along $-\mathbf{w}/\|\mathbf{w}\|$, the unit vector across it. After a distance t you stand at $\mathbf{x} - t\,\mathbf{w}/\|\mathbf{w}\|$; you are on the line when

$$\mathbf{w}^\top\left(\mathbf{x} - t\,\tfrac{\mathbf{w}}{\|\mathbf{w}\|}\right) + b = 0 \implies \boxed{t = d(\mathbf{x}) = \frac{\mathbf{w}^\top\mathbf{x} + b}{\|\mathbf{w}\|}}$$

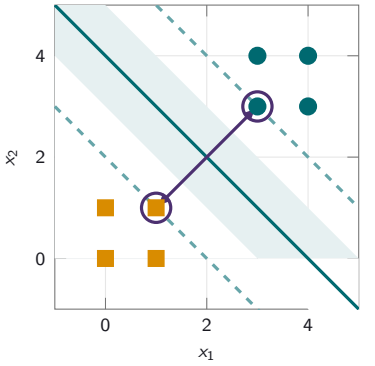
since $\mathbf{w}^\top\mathbf{w}/\|\mathbf{w}\| = \|\mathbf{w}\|$. *Distance* = *score* $\div\ \|\mathbf{w}\|$; the sign says which side.

Worked: $\mathbf{x} = (4, 3)$ and the line $x_1 + x_2 = 4$

score $= 4 + 3 - 4 = 3$, $\|\mathbf{w}\| = \sqrt{1^2 + 1^2} = 1.414$, so $d = 3/1.414 = 2.12$. Foot: $(4, 3) - 2.12\,(0.707, 0.707) = (2.5, 1.5)$; check $2.5 + 1.5 = 4$. For $(1, 1)$: score -2, $d = -1.41$, other side.

Scale $(\mathbf{w}, b)$ by 3: score 9, $\|\mathbf{w}\| = 4.24$, ratio still 2.12. Distance belongs to the line, not to $(\mathbf{w}, b)$.

The Margin Of A Line Is Its Distance To The Nearest Point



arrows: the two smallest distances, 1.414 each; the band is the corridor they define

A spam point should have $d > 0$, a non-spam point $d < 0$. Multiply by the label: $y_i\, d(\mathbf{x}_i)$ is positive exactly when point i is on its correct side. For the line $x_1 + x_2 = 4$:

i	1	2	3	4	5	6	7	8
score	2	3	3	4	−2	−3	−3	−4
$d = \text{score}/\lVert\mathbf{w}\rVert$	1.41	2.12	2.12	2.83	−1.41	−2.12	−2.12	−2.83
$y_i\, d(\mathbf{x}_i)$	**1.41**	2.12	2.12	2.83	**1.41**	2.12	2.12	2.83

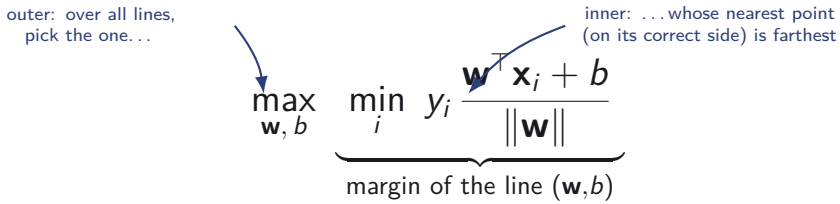
Margin

The margin of a separating line is the smallest of these numbers,

$$\text{margin}(\mathbf{w}, b) = \min_i \; y_i \, \frac{\mathbf{w}^\top \mathbf{x}_i + b}{\lVert\mathbf{w}\rVert}.$$

For $x_1 + x_2 = 4$ it is 1.414, attained by points 1 and 5, the circled ones.

The Goal As A Formula: The Line With The Largest Margin



This is the whole idea, written down once. Every later formula is a rewriting of this one, chosen to make it easier to solve.

If the minimum is negative, some point is on the wrong side; a line with a positive maximum separates the data.

Why not solve it as it stands?

- A min inside a max: the objective has kinks wherever the nearest point changes.
- The ratio makes it non-linear in $(\mathbf{w}, b)$.
- Many pairs $(\mathbf{w}, b)$ describe the same line, so the maximiser is not unique.

Two clean-up steps fix all three.

Clean-Up Step 1: One Line, Many $(\mathbf{w}, b)$. We Choose The Scale

The pairs $(\mathbf{w}, b)$, $(3\mathbf{w}, 3b)$ and $(\frac{1}{2}\mathbf{w}, \frac{1}{2}b)$ all satisfy $\mathbf{w}^\top \mathbf{x} + b = 0$ at the same points: *one line, infinitely many equations*. Slide 7 showed the distance does not care which one we use. So we may pick the most convenient one.

Our choice of scale

Scale $(\mathbf{w}, b)$ so that the *nearest* point scores exactly 1:

$$\min_i \; y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1.$$

Then the margin is $\min_i y_i(\mathbf{w}^\top \mathbf{x}_i + b)/\|\mathbf{w}\| = 1/\|\mathbf{w}\|$.

The messy inner minimum has become the single number $1/\|\mathbf{w}\|$.

$(\mathbf{w}, b)$	$\|\mathbf{w}\|$	nearest score	margin
$(1, 1), -4$	1.414	2	$2/1.414 = 1.414$
$(\frac{1}{2}, \frac{1}{2}), -2$	0.707	**1**	$1/0.707 = 1.414$
$(\frac{1}{4}, \frac{1}{4}), -1$	0.354	0.5	$0.5/0.354 = 1.414$

Insight

Three equations, one line, one margin. Only the middle row has nearest score 1; that is the one we keep, and in it the margin is $1/\|\mathbf{w}\|$. From now on "wide margin" means "short $\mathbf{w}$".

Clean-Up Step 2: Shortest w, Every Point At Least One Unit Away

Maximising $1/\|\mathbf{w}\|$ is minimising $\|\mathbf{w}\|$, i.e. minimising $\frac{1}{2}\|\mathbf{w}\|^2$ (same minimiser, no square root, the $\frac{1}{2}$ cancels a 2 later). And "nearest score $= 1$" is the same as "every score ≥ 1".

The optimisation problem (hard margin)

$$\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \qquad \text{subject to} \qquad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \text{for every point } i = 1, \ldots, n.$$

Reading it.

- Objective $\frac{1}{2}\|\mathbf{w}\|^2$: "make the margin $1/\|\mathbf{w}\|$ wide".
- Constraint i: "point i is on its correct side, with score at least 1, i.e. distance at least $1/\|\mathbf{w}\|$: outside the corridor".
- Unknowns: $\mathbf{w}$ and b only. The data $\mathbf{x}_i, y_i$ are fixed numbers.

Naming. This is the **primal** problem: stated in the original unknowns $(\mathbf{w}, b)$. An equivalent problem in different unknowns, the **dual**, comes after the Lagrangian. "Hard margin": no point may be inside the corridor; relaxed in the soft-margin section.

Shape. A quadratic bowl over a region cut out by n half-spaces. The bowl $\frac{1}{2}\|\mathbf{w}\|^2$ is strictly convex (perfectly round), so the best $\mathbf{w}$ is unique, and the active constraints then fix b.

The Same Problem With The Numbers Filled In

Three unknowns, w_1, w_2, b. Eight constraints, one per point ($y_i = +1$ keeps the sign, $y_i = -1$ flips it):

$$
\begin{aligned}
3w_1 + 3w_2 + b &\geq 1 \quad (3,3),\ y = +1 \\
4w_1 + 3w_2 + b &\geq 1 \quad (4,3) \\
3w_1 + 4w_2 + b &\geq 1 \quad (3,4) \\
4w_1 + 4w_2 + b &\geq 1 \quad (4,4) \\
-(w_1 + w_2 + b) &\geq 1 \quad (1,1),\ y = -1 \\
-(w_2 + b) &\geq 1 \quad (0,1) \\
-(w_1 + b) &\geq 1 \quad (1,0) \\
-b &\geq 1 \quad (0,0)
\end{aligned}
$$

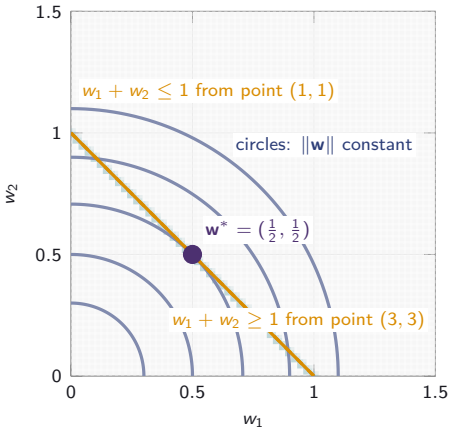
Minimise $\frac{1}{2}(w_1^2 + w_2^2)$ over the region they cut out.

- Quadratic objective, linear constraints: a **quadratic program**. Mature theory, off-the-shelf solvers.
- Add the first and fifth: $2w_1 + 2w_2 \geq 2$, so $w_1 + w_2 \geq 1$.
- The shortest $\mathbf{w}$ with $w_1 + w_2 \geq 1$ is $w_1 = w_2 = \frac{1}{2}$, and with it the fifth constraint gives $b \leq -2$, the first $b \geq -2$: so $b = -2$.
- Check all eight: scores $1, 1.5, 1.5, 2,\ 1, 1.5, 1.5, 2$. All ≥ 1. Margin $1/\|\mathbf{w}\| = 1.414$.

Insight

Two of eight constraints decided everything. The next slide draws that.

The Problem As A Picture: A Bowl Over The Feasible Region



the (w_1, w_2) plane at $b = -2$: feasible set (teal, here a segment), level circles of $\frac{1}{2}\|\mathbf{w}\|^2$, and the optimum

Freeze $b = -2$ so the picture has two axes. Each constraint is a half-plane in (w_1, w_2). The half-planes of $(3, 3)$ and $(1, 1)$ face each other across the *same* line $w_1 + w_2 = 1$, so the feasible set collapses to a segment.

The objective $\frac{1}{2}\|\mathbf{w}\|^2$ is a bowl centred at the origin; its level sets are circles. The lowest feasible point is where the smallest circle just touches the segment: the midpoint $(\frac{1}{2}, \frac{1}{2})$, radius 0.707.

Two constraints are **active** (hold with equality); the other six are **slack** (hold with room to spare). The active ones belong to the circled points of slide 8. The next four slides prove that this is no coincidence, and show how to find the active constraints without drawing.

Why Go Further? What Pricing The Constraints Buys Us

We already solved the eight-point problem by hand. Three reasons not to stop here:

1. **To solve it in general.** With n points in d dimensions, we cannot "add the first and fifth". We need a systematic way to find which constraints are active.
2. **To see which points matter.** The standard tool for constrained minimisation, the Lagrangian, attaches a number $\alpha_i \geq 0$ to every constraint. It will turn out that $\alpha_i > 0$ exactly for the points that touch the corridor.
3. **To open the door to kernels.** Rewriting the problem in terms of the α_i (the *dual* problem) makes the data appear only through inner products $\mathbf{x}_i^\top \mathbf{x}_j$. That single fact is what makes curved boundaries possible later.

Primal and dual, in words

Primal: minimise over the line's own parameters $(\mathbf{w}, b)$, subject to the n point constraints.
Dual: maximise over the n constraint prices $\boldsymbol{\alpha}$, subject to simple sign and sum conditions.
For this problem the two optimal values are equal, and each solution can be read off from the other.

Insight

Read α_i as the *price* of point i: how much the objective $\frac{1}{2}\|\mathbf{w}\|^2$ would fall if point i's constraint were loosened by one unit. A point far from the corridor has price zero, because loosening its constraint changes nothing.

The Lagrangian: A Price On Every Constraint

Attach a multiplier $\alpha_i \geq 0$ to constraint i and subtract "price times slack" from the objective:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \big[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 \big].$$

Stationarity in $\mathbf{w}$ and b at the solution:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i = 0 \;\Rightarrow\; \boxed{\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \boxed{\sum_i \alpha_i y_i = 0}$$

Complementary slackness (the key condition):

$$\alpha_i \big[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 \big] = 0 \quad \text{for every } i.$$

Either the constraint is active (score exactly 1: the point touches the corridor) or its price is zero. A point with room to spare costs nothing and contributes nothing to $\mathbf{w}$.

Summary

$\mathbf{w}$ is a weighted sum of data points, and only points on the corridor's edge get non-zero weight: the **support vectors**.

Two Players, One Table: Who Moves First Matters

In one sentence: Before the dual, one idea: when two players choose in turn, the order can change the result.

You pick a model (a row). A tester picks the test condition (a column). Each entry is your test accuracy in per cent. You want it high; the tester wants it low.

	clean	noisy	row min
model A (simple)	80	75	**75**
model B (complex)	95	60	60
column max	95	**75**	

Row min: the worst the tester can do to you in that row. Column max: the best you can do in that column.

You move first. Whatever row you pick, the tester answers with the column that hurts you most: the smallest entry in your row. Model A gives 75, model B gives 60. So you pick A:

$$\max_{\text{row}} \min_{\text{column}} a = \max(75, 60) = 75.$$

The tester moves first. Whatever column they pick, you answer with your best model: the largest entry in that column. Clean gives 95, noisy gives 75. So the tester picks noisy:

$$\min_{\text{column}} \max_{\text{row}} a = \min(95, 75) = 75.$$

Summary

Here both orders give 75, at (model A, noisy data). That entry is the smallest in its row and the largest in its column. It is called a **saddle point**.

Why The Inside Is Computed First

In $\max_{\text{row}} \min_{\text{column}}$ the *outside* symbol belongs to the player who moves first (you), and the *inside* symbol to the player who answers (the tester). Yet we always compute the inside first. Why?

Because the first mover thinks ahead

Before you choose a row, you ask: "if I pick this row, what will the tester do?" The answer to that question is the inner min. You cannot judge a row until you know the reply it will get.

1. **Inside (look ahead):** for every row, find the tester's reply.
   A: $\min(80, 75) = 75$. B: $\min(95, 60) = 60$.
2. **Outside (decide):** pick the row whose reply is best for you: $\max(75, 60) = 75$.

Insight

Chess works the same way. You move first, but you choose a move by imagining your opponent's best answer to each candidate. Order of *moves*: outside first. Order of *computing*: inside first.

Warning

The tempting slip: grab the row with the biggest entry (model B, 95) without looking ahead. The tester answers "noisy" and you get 60, less than the 75 that model A guarantees.

When The Order Changes The Answer

A second table, same rules ("shifted": test data drawn from a different population):

	clean	shifted	row min
model A	90	60	60
model B	70	80	**70**
column max	90	**80**	

$$\max_{\text{row}} \min_{\text{column}} = 70 \; < \; 80 = \min_{\text{column}} \max_{\text{row}}.$$

No entry is the smallest in its row and the largest in its column: there is no saddle point.

Moving second never hurts. The second player sees the first choice and answers it. So for every table

$$\max_{\text{row}} \min_{\text{column}} a \;\leq\; \min_{\text{column}} \max_{\text{row}} a.$$

One line of proof: for any row r and any column c,

$$\min_{c'} a(r, c') \;\leq\; a(r, c) \;\leq\; \max_{r'} a(r', c).$$

The left end depends only on r and the right end only on c, so the largest left end is at most the smallest right end.

Insight

This inequality is called **weak duality**. When the two sides are equal, as in the first table, we have **strong duality**.

The Same Game In The SVM: The Line Against The Prices

Two players again. The line $(\mathbf{w}, b)$ wants $\mathcal{L}$ small; the prices $\boldsymbol{\alpha} \geq 0$ want it large:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \tfrac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i \big[y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1\big].$$

The line moves first (the primal), $\min_{\mathbf{w},b} \max_{\boldsymbol{\alpha}\geq 0} \mathcal{L}$. Look ahead for one fixed line:

- If some point breaks its constraint, its bracket is negative, and a huge price α_i sends $\mathcal{L}$ to $+\infty$.
- If every constraint holds, the best reply is $\boldsymbol{\alpha} = \mathbf{0}$, and then $\mathcal{L} = \tfrac{1}{2}\|\mathbf{w}\|^2$.

So the line must obey every constraint and then make $\tfrac{1}{2}\|\mathbf{w}\|^2$ small: exactly our problem.

The prices move first (the dual), $\max_{\boldsymbol{\alpha}\geq 0} \min_{\mathbf{w},b} \mathcal{L}$. Now the inside is a plain minimisation with no constraints: set the derivatives to zero. That is what the next slide does.

Summary

For the SVM both orders give the same value, like the first table: the bowl is convex and the constraints are straight lines. So we may solve whichever order is easier, and the dual is the one that kernels fit into.

The Same Example In Hindi

दो खिलाड़ी, एक तालिका: कौन पहले चलता है?

आप एक model चुनते हैं (एक row)। Tester test की स्थिति चुनता है (एक column)। हर खाने में आपकी test accuracy (%) है। आप इसे बड़ा चाहते हैं, tester छोटा।

	साफ़ data	noisy data	आपके लिए सबसे बुरा
model A (सरल)	80	75	**75**
model B (जटिल)	95	60	60
आपके लिए सबसे अच्छा	95	**75**	

आप पहले चलते हैं (max-min). चुनने से पहले हर row के लिए सोचिए: "अगर मैं यह row चुनूँ, तो tester क्या करेगा?" वह उस row की सबसे छोटी संख्या चुनेगा: A $\to$ 75, B $\to$ 60। फिर इनमें से सबसे अच्छी चुनिए: $\max(75,\ 60) = 75$।

अंदर से पहले क्यों? चाल बाहर वाला खिलाड़ी (आप) पहले चलता है, पर चाल चलने से पहले वह आगे सोचता है। वही "आगे सोचना" अंदर वाला min है। इसलिए हिसाब अंदर से शुरू होता है। Chess में भी ऐसा ही होता है: चाल आप पहले चलते हैं, पर हर चाल के लिए सोचते हैं कि सामने वाला क्या जवाब देगा।

Tester पहले चलता है (min-max). हर column में आप सबसे बड़ी संख्या चुनेंगे: 95 और 75। Tester इनमें सबसे छोटी चुनेगा: $\min(95,\ 75) = 75$।

दोनों तरफ़ 75 आया। (model A, noisy data) वाला 75 अपनी row में सबसे छोटा और अपने column में सबसे बड़ा है। इसे **saddle point** कहते हैं। SVM में भी दोनों क्रम एक ही मान देते हैं, इसीलिए हम dual हल कर सकते हैं।

The Dual Problem: Optimise The Prices Instead Of The Line

Put $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and $\sum_i \alpha_i y_i = 0$ into $\mathcal{L}$. The b term vanishes and $\frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i y_i \mathbf{w}^\top \mathbf{x}_i = -\frac{1}{2}\|\mathbf{w}\|^2$, leaving

The dual problem (hard margin)

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j \quad \text{subject to} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0.$$

- n unknowns (one price per point) instead of $d+1$; still a quadratic program, concave in $\boldsymbol{\alpha}$: one maximum.
- Strong duality holds (convex primal with a strictly feasible point): the primal and dual optimal values are *equal*.
- Recover the line from the prices: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$, and b from any point with $\alpha_i > 0$.

Warning

Look at where the data enter: only through $\mathbf{x}_i^\top \mathbf{x}_j$, the inner products. Nothing else about the coordinates of $\mathbf{x}_i$ is used. Hold on to this; it is the door the kernel walks through in the kernel section.

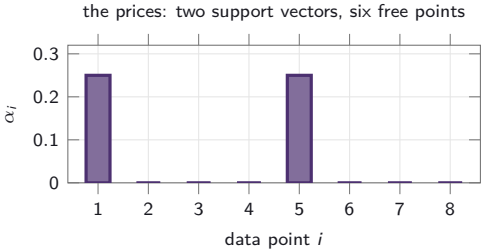
Solving The Dual By Hand On The Running Example

Guess (and then verify) that only $(3,3)$ and $(1,1)$ have non-zero price: $\alpha_1 = \alpha_5 = \alpha$, all others 0. The equality $\alpha_1 y_1 + \alpha_5 y_5 = \alpha - \alpha = 0$ holds for any α.

$$\mathbf{w} = \alpha\,(+1)(3,3) + \alpha\,(-1)(1,1) = \alpha\,(2,2).$$

Both points touch the corridor: $\mathbf{w}^\top \mathbf{x}_1 + b = 1$ and $\mathbf{w}^\top \mathbf{x}_5 + b = -1$. Subtract: $\mathbf{w}^\top(\mathbf{x}_1 - \mathbf{x}_5) = 2$, i.e. $\alpha\,(2,2)\cdot(2,2) = 8\alpha = 2$, so

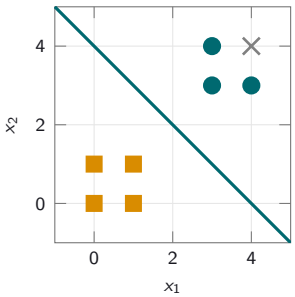
$$\boxed{\alpha = \tfrac{1}{4}}, \qquad \mathbf{w} = (\tfrac{1}{2}, \tfrac{1}{2}), \qquad b = 1 - \mathbf{w}^\top \mathbf{x}_1 = 1 - 3 = -2.$$



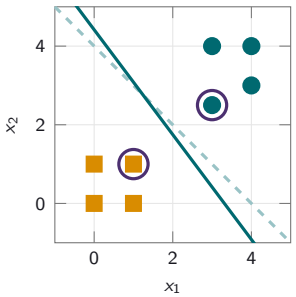
Verify. All eight scores $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ are $1, 1.5, 1.5, 2,\ 1, 1.5, 1.5, 2$: every KKT condition holds, and uniqueness makes this *the* answer.
Duality check. Primal $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{4}$; dual $\frac{1}{2} - \frac{1}{4} = \frac{1}{4}$. Equal, as promised.

Only The Support Vectors Matter

drop $(4, 4)$: nothing moves



move $(3, 3) \to (3, 2.5)$: the line turns



Left: removing a point with $\alpha_i = 0$ changes nothing; the solution is $(\frac{1}{2}, \frac{1}{2}), -2$ again.

Right: nudging a support vector changes the line to $\mathbf{w} = (0.64, 0.48)$, $b = -2.12$.

Summary

The classifier is a function of a handful of points. Compare logistic regression, where every point pulls on the coefficients. Sparsity in the data, not in the features.

Interactive explorer: drag a non-support point (nothing happens), then a support vector ↗

Classifying A New Point: A Sum Over Support Vectors

With $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$, the score of a new point $\mathbf{x}$ is

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{i \in \mathrm{SV}} \alpha_i y_i \, \mathbf{x}_i^\top \mathbf{x} + b, \qquad \widehat{y} = \operatorname{sign} f(\mathbf{x}).$$

On the running example, for a new email $\mathbf{x} = (2, 3)$:

$$f = \tfrac{1}{4}(3,3)\cdot(2,3) - \tfrac{1}{4}(1,1)\cdot(2,3) - 2 = \tfrac{15}{4} - \tfrac{5}{4} - 2 = 0.5,$$

positive, so "spam". Its distance from the line is $0.5/\|\mathbf{w}\| \approx 0.71$: halfway to the corridor's edge, which is 1.41 away.

Insight

Two things to notice, both used later:

1. f is a signed distance (times $\|\mathbf{w}\|$), *not a probability*. Thresholds and costs from the evaluation lecture still apply; a probability needs an extra calibration step.
2. f touches $\mathbf{x}$ only through $\mathbf{x}_i^\top \mathbf{x}$: inner products with the support vectors.

The Same Numbers In scikit-learn

```
from sklearn.svm import SVC
svm = SVC(kernel="linear", C=1e6)  # huge C: hard margin
svm.fit(X, y)                      # the eight emails

svm.support_vectors_   # [[1, 1], [3, 3]]
svm.dual_coef_         # [[-0.25, 0.25]]
svm.intercept_         # [-2.]
svm.decision_function([[2, 3]])    # [0.5]
```

Each attribute is a symbol from the last few slides:

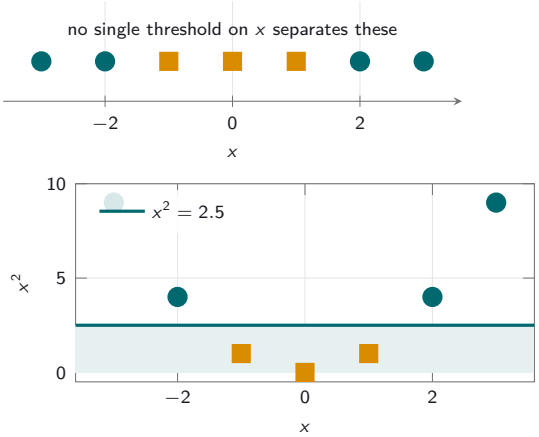
attribute	symbol
`support_vectors_`	the $\mathbf{x}_i$ with $\alpha_i > 0$
`dual_coef_`	$\alpha_i y_i$: price times label
`intercept_`	b
`decision_function`	$f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$

Check the last line by hand:
$0.25\,(3,3)\cdot(2,3) - 0.25\,(1,1)\cdot(2,3) - 2 = 0.5$, the same score as the previous slide.

Warning

`dual_coef_` stores $\alpha_i y_i$, not α_i: its sign is the label. The prices are its absolute values, $\alpha_1 = \alpha_5 = \frac{1}{4}$.

When No Line Works: One Dimension



Seven sensor readings, each measured as its distance x from the usual value: anomalies at $x = \pm 2, \pm 3$, normal readings at $x = 0, \pm 1$. An anomaly can sit on either side, so no single threshold on x separates them; the hard-margin problem has *no feasible point*.

Add a coordinate you can compute from x: $\phi(x) = (x,\ x^2)$. In the new plane the anomalies sit high and the normal readings low, and a horizontal line $x^2 = 2.5$ separates them with a corridor to spare. Support vectors: $x = \pm 1$ and $x = \pm 2$.

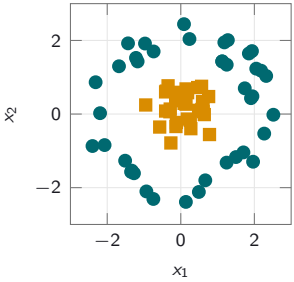
Read the line back: $x^2 = 2.5$ means $|x| = 1.58$, i.e. *two* thresholds. A line in the lifted space is a curve in the original one.

Insight

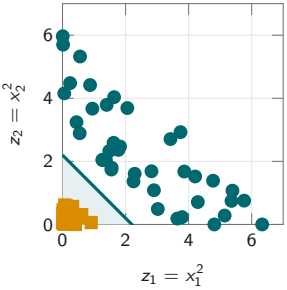
We did not change the algorithm at all. We changed what we handed it.

When No Line Works: Two Dimensions

inside vs outside: no line separates them



after squaring the coordinates: a line does it



30 normal readings inside a radius of 1, 40 anomalies in a ring beyond 1.8. Lift with $\phi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2)$. The plane $z_1 + z_2 = 2.2$ separates them; in the original coordinates that is the circle $x_1^2 + x_2^2 = 2.2$. Squares and cross-terms turn every conic into a hyperplane. **The linear machine has not changed; its input has.**

The Cost Of Lifting Explicitly

All monomials up to degree d in D input features number $\binom{D+d}{d}$:

inputs D	degree 2	degree 3
2	6	10
10	66	286
100	5,151	176,851
1,000	501,501	167,668,501

A thousand raw features and cubic terms: 1.7×10^8 coordinates per point, before a single inner product is taken.

Warning

Explicit lifting is a dead end at scale. Storing $\phi(\mathbf{x}_i)$ for n points costs $n \times \binom{D+d}{d}$ numbers, and every $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ costs that many multiplications.

Question

The dual problem and the decision function only ever ask for $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$. Do we need ϕ itself?

The Kernel Trick: The Inner Product Without The Lift

In one sentence: A kernel is a function $K(\mathbf{x}, \mathbf{z})$ that equals $\phi(\mathbf{x})^\top \phi(\mathbf{z})$ for some lift ϕ; computing K costs about as much as one inner product in the original space.

For the quadratic lift $\phi(\mathbf{x}) = (1,\ \sqrt{2}x_1,\ \sqrt{2}x_2,\ x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2)$, expand

$$(1 + \mathbf{x}^\top \mathbf{z})^2 = (1 + x_1z_1 + x_2z_2)^2 = 1 + 2x_1z_1 + 2x_2z_2 + x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2 = \phi(\mathbf{x})^\top \phi(\mathbf{z}).$$

Six-dimensional inner product, two multiplications and a square.

kernel	formula	lifts to
linear	$\mathbf{x}^\top \mathbf{z}$	nothing
polynomial	$(1 + \mathbf{x}^\top \mathbf{z})^d$	all monomials to degree d
Gaussian (RBF)	$\exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)$	infinitely many

Replace every $\mathbf{x}_i^\top \mathbf{x}_j$ in the dual by $K(\mathbf{x}_i, \mathbf{x}_j)$ and every $\mathbf{x}_i^\top \mathbf{x}$ in f by $K(\mathbf{x}_i, \mathbf{x})$:

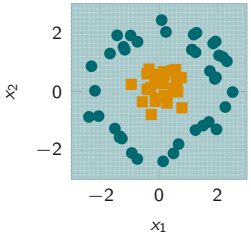
$$f(\mathbf{x}) = \sum_{i \in \mathrm{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b.$$

$\mathbf{w}$ itself is never formed. It may live in a space we cannot write down.

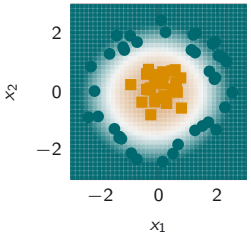
Check: $\mathbf{x} = (1, 2)$, $\mathbf{z} = (0.5, -1)$ gives $(1 + \mathbf{x}^\top \mathbf{z})^2 = 0.25 = \phi(\mathbf{x})^\top \phi(\mathbf{z})$.

Three Kernels, One Dataset

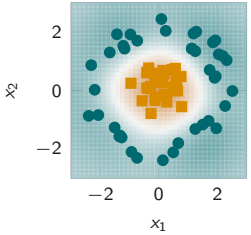
linear: 70/70 SVs, 57% correct



$(1 + \mathbf{x}^\top \mathbf{z})^2$: 7 SVs, all correct



Gaussian $\gamma = 0.5$: 12 SVs, all correct



Colour is the decision function $f(\mathbf{x})$: teal positive, orange negative, white at the boundary $f = 0$. Same solver, same $C = 10$; only the kernel changed.

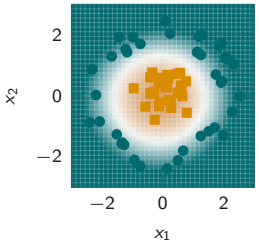
Insight

The linear machine gives up: every point becomes a support vector. The quadratic kernel finds the circle with seven points; the Gaussian with twelve.

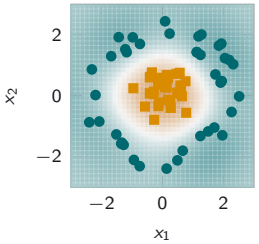
Interactive explorer: switch kernels on the rings and on your own points ↗

The Gaussian Kernel Is A Similarity, And γ Sets Its Reach

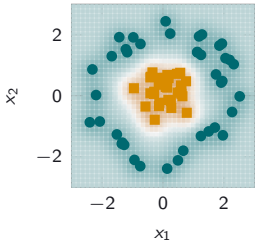
$\gamma = 0.05$: 17 SVs, too smooth



$\gamma = 0.5$: 12 SVs



$\gamma = 5$: 47 SVs, islands



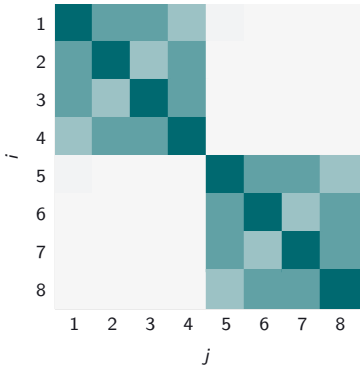
$K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$ is 1 at $\mathbf{z} = \mathbf{x}$ and decays with distance: a *similarity*. The decision function is a sum of bumps of height $\alpha_i y_i$ centred on the support vectors, with width set by γ.

Warning

Large γ means narrow bumps: the boundary hugs individual points and the support-vector count climbs toward n. That is overfitting, and γ (like C) must be chosen on validation data, never on training data.

What Makes A Function A Kernel: The Gram Matrix

Gaussian Gram matrix K_{ij}, $\gamma = 0.5$, the eight points



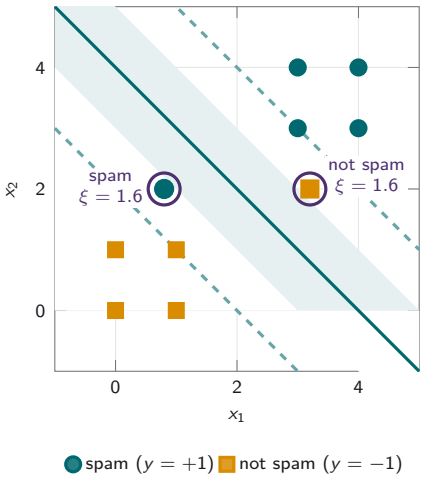
The dual only needs the $n \times n$ table $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$, the **Gram matrix**. For the eight points under the Gaussian kernel it is block-like: points within a class are similar (dark), across classes nearly 0 (white).

For the dual to be a well-posed maximisation, the quadratic term $\sum_{ij} \alpha_i \alpha_j y_i y_j K_{ij}$ must be ≥ 0 for every $\boldsymbol{\alpha}$: K must be **positive semidefinite**. Mercer's theorem says exactly those functions are inner products in *some* lifted space, so any positive semidefinite similarity you can write down is a legal kernel, whether or not you can write its ϕ.

Insight

The Gram matrix is also the cost: n^2 entries. With $n = 10^5$ points that is 10^{10} numbers, which is why kernel SVMs are a mid-size-data tool and linear SVMs are the large-data tool.

Two Intruders, And No Line That Separates Everything



A normal email at $(3.2, 2)$ and a spam email at $(0.8, 2)$. The old line puts both on the wrong side, and no line puts all ten points outside a corridor. The hard-margin problem has no feasible point, and this time no honest lift will save it.

Let each point buy its way out with a **slack** $\xi_i \geq 0$:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i.$$

$\xi_i = 0$: on or outside the corridor's edge, fine. $0 < \xi_i < 1$: inside the corridor but on the correct side. $\xi_i = 1$: exactly on the line. $\xi_i > 1$: on the wrong side. On the old line both intruders have $\xi = 1.6$.

Soft Margin: Pay For Every Point In The Wrong Place

In one sentence: Keep the margin objective, add the slacks to the constraints, and charge C per unit of slack.

Soft-margin SVM

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

- Still a quadratic program, still convex, always feasible: ξ_i large enough satisfies anything.
- C is the exchange rate between corridor width and violations. $C \to \infty$ recovers the hard margin; $C \to 0$ makes $\mathbf{w} \to \mathbf{0}$.
- The slacks are not free parameters to tune; at the optimum each is exactly the point's shortfall, $\max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))$.

Insight

Two prices in one problem: $\frac{1}{2}\|\mathbf{w}\|^2$ charges a narrow corridor, $C\sum \xi_i$ charges intrusions. Every choice of C is a different line, and the next slide shows three.

Slack In Numbers: Six Emails, One Feature, Two Candidate Lines

One feature x per email (say, a count of suspicious words). Not spam ($y = -1$) at $x = 0,\ 1,\ 2.5$; spam ($y = +1$) at $x = 3,\ 5,\ 6$. Two candidate lines $f(x) = wx + b$: **wide** $f_A(x) = 0.5x - 1.5$ (boundary $x = 3$, corridor $1 \le x \le 5$) and **narrow** $f_B(x) = 4x - 11$ (boundary $x = 2.75$, corridor $2.5 \le x \le 3$). For each point, $\xi_i = \max\big(0,\ 1 - y_i f(x_i)\big)$:

x_i	y_i	$f_A(x_i)$	$y_i f_A(x_i)$	ξ_i (wide)	$f_B(x_i)$	$y_i f_B(x_i)$	ξ_i (narrow)
0	-1	-1.5	1.5	0	-11	11	0
1	-1	-1	1	0	-7	7	0
2.5	-1	-0.25	0.25	**0.75**	-1	1	0
3	$+1$	0	0	**1**	1	1	0
5	$+1$	1	1	0	9	9	0
6	$+1$	1.5	1.5	0	13	13	0
			total slack	1.75			0

Read the two non-zero slacks on the wide line. $x = 2.5$: $\xi = 0.75$, inside the corridor but on the correct side ($0 < \xi < 1$). $x = 3$: $\xi = 1$, exactly on the line. A point on the wrong side would have $\xi > 1$. The narrow line leaves every point on or beyond its corridor's edge, so all its slacks are 0.

Which Line Wins Depends On The Price C

The soft-margin objective adds the two costs, $\frac{1}{2}w^2 + C\sum_i \xi_i$:

	corridor width $2/\lvert w\rvert$	$\frac{1}{2}w^2$	$\sum_i \xi_i$	objective $\frac{1}{2}w^2 + C\sum_i \xi_i$
wide, $w = 0.5$	4	0.125	1.75	$0.125 + 1.75\,C$
narrow, $w = 4$	0.5	8	0	8

$C = 0.1$. Wide: $0.125 + 1.75 \times 0.1 = 0.300$. Narrow: 8. The wide line wins: a violation is cheap, so it keeps the wide corridor and accepts two intrusions.

$C = 10$. Wide: $0.125 + 1.75 \times 10 = 17.625$. Narrow: 8. The narrow line wins: violations are dear, so it squeezes in between 2.5 and 3.

The switch point. The two costs are equal when

$$0.125 + 1.75\,C = 8 \quad \Longrightarrow \quad C = \frac{7.875}{1.75} = 4.5.$$

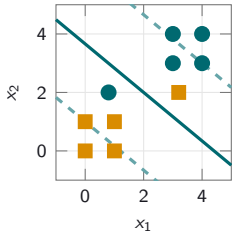
Below $C = 4.5$ the wide line is cheaper; above it, the narrow one.

Warning

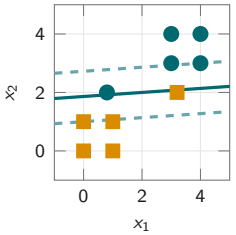
These are two candidates chosen to show the trade-off, not the best line for each C. The solver searches every line, but it weighs width against slack in exactly this way.

C Sets The Price Of A Violation

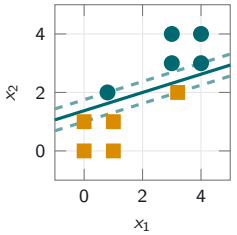
$C = 0.1$: margin 2.05, 4 violators



$C = 1$: margin 0.86, 2 violators



$C = 10$: margin 0.36, 0 violators



Small C: violations are cheap; the machine keeps the wide diagonal corridor and writes the intruders off. Large C: violations are dear; it twists the line to get every point right, at a corridor almost six times narrower (0.36 against 2.05).

Warning

The $C = 10$ line separates the training data and would be the worst of the three on new emails. Choose C by validation error, as for Ridge.

The Same Problem As A Loss Plus A Penalty

In one sentence: At the optimum $\xi_i = \max(0,\, 1 - y_i f(\mathbf{x}_i))$, so the soft-margin SVM is hinge loss plus the Ridge penalty you already know.

Eliminate the slacks: the smallest ξ_i allowed is $\max(0, 1 - y_i f(\mathbf{x}_i))$. Then the whole problem is

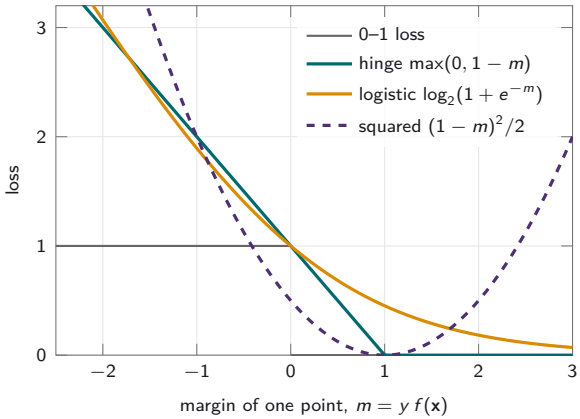
$$\min_{\mathbf{w},b} \sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big) + \frac{1}{2C}\|\mathbf{w}\|^2.$$

- The first term is the **hinge loss**: zero once a point is beyond the corridor.
- The second is the Ridge penalty with $\lambda = 1/C$.

Summary

Logistic regression and the SVM are one recipe, loss + penalty, with two different losses. The corridor picture and the loss picture are one object seen from two sides.

Four Losses On The Margin Axis



$m = y\, f(\mathbf{x})$ is positive when the point is on the correct side and larger than 1 when it is beyond the corridor.

- **Hinge** is exactly zero for $m \geq 1$: those points drop out of $\mathbf{w}$. That is where support vectors come from.
- **Logistic** is never zero: every point keeps pulling, so logistic regression has no support vectors.
- Both are convex upper bounds on the **0–1 loss**, the count of mistakes, which is not convex and cannot be optimised directly.
- **Squared** loss punishes being *too* right; that is why least squares is a poor classifier.

The Soft-Margin Dual: One Extra Wall

Repeating the Lagrangian with the slack constraints (multipliers $\mu_i \geq 0$ for $\xi_i \geq 0$) gives $C - \alpha_i - \mu_i = 0$, hence $\alpha_i \leq C$, and the slacks vanish from the objective:

Soft-margin SVM (dual)

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

Three kinds of point now, read off α_i:

$\alpha_i = 0$	$y_i f \geq 1$	outside the corridor, no influence
$0 < \alpha_i < C$	$y_i f = 1$	on the corridor's edge
$\alpha_i = C$	$y_i f \leq 1$	inside the corridor or wrong; capped

Computing b. Use a point with $0 < \alpha_i < C$: it sits exactly on the edge, $y_i f(\mathbf{x}_i) = 1$, so $b = y_i - \sum_j \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i)$. A point with $\alpha_i = C$ may be inside the corridor and would give the wrong b.

Insight

The cap is the whole difference from the hard margin: no single point can buy unlimited influence. At $C = 0.1$ the intruders are stuck at $\alpha = 0.1$ and can only tilt the line slightly.

Algorithmic Thinking: What Does It Cost To Solve?

In one sentence: Several algorithms solve the same problem; which to use depends on n, d, and whether there is a kernel.

solver	unknowns	per step	memory	use when
generic QP, primal	$d+1$	interior point, $O((d+n)^3)$	$O(nd)$	small n, no kernel
generic QP, dual	n	$O(n^3)$	$O(n^2)$ Gram	tiny n; for understanding
SMO, dual	n, two at a time	$O(n)$ per pair	$O(n^2)$ Gram, or recompute rows	kernels, $n \lesssim 10^5$ (libsvm)
subgradient, primal (Pegasos)	$d+1$	$O(d)$ per sample	$O(d)$	linear, huge n (SGD)
coordinate descent, dual	n	$O(d)$ per coordinate	$O(nd)$	linear, large n (liblinear)

What Each Solver Exploits

The problem is one. The algorithms differ in which structure they lean on:

- **SMO**: the dual's constraints couple the α_i only through one equality, so two variables at a time can be solved exactly.
- **Stochastic subgradient**: the primal objective is a sum over samples, so one sample gives an unbiased step.
- **Coordinate descent**: with a linear kernel, one α_i changes $\mathbf{w}$ by a rank-one update in $O(d)$.
- All three exploit **sparsity**: most α_i end at 0 and most samples are beyond the corridor.

Warning

The Gram matrix is the wall. Kernel methods scale with n^2; when n is a million you go linear, approximate the kernel with a finite lift (random features), or subsample.

Insight

Notice what a library name encodes: libsvm is SMO on the dual with kernels; liblinear is coordinate descent or Newton on the primal without them. The choice of solver is the choice of what you are willing to pay for.

SMO: Why Two Multipliers At A Time

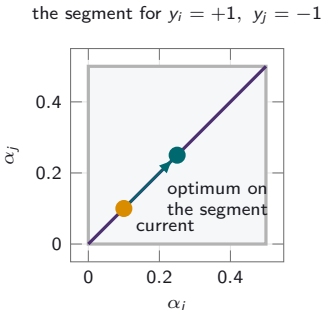
Coordinate ascent on the dual, one α_i at a time, is impossible: the equality $\sum_i \alpha_i y_i = 0$ fixes α_i once the others are fixed. So take **two** at a time. With all others frozen,

$$\alpha_i y_i + \alpha_j y_j = \text{const}$$

is a line segment in the (α_i, α_j) plane (clipped to $[0, C]^2$), and the dual objective along it is a one-variable quadratic. Its maximum is in closed form:

$$\alpha_j^{\text{new}} = \alpha_j - \frac{y_j(E_i - E_j)}{\eta}, \qquad \eta = 2K_{ij} - K_{ii} - K_{jj} \le 0,$$

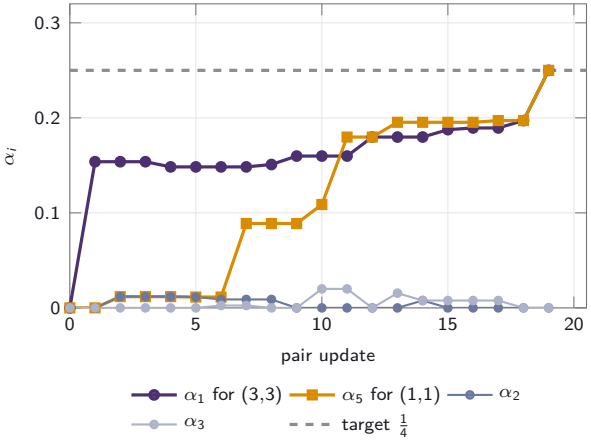
where $E_i = f(\mathbf{x}_i) - y_i$ is the current error on point i; then clip to the segment and set α_i from the constraint.



Summary

Sequential Minimal Optimisation: pick a pair that violates the KKT conditions, solve their two-variable problem exactly, repeat. No matrix factorisation, no line search, only kernel evaluations.

SMO On The Running Example



A plain SMO with random partner choice, started from $\boldsymbol{\alpha} = \mathbf{0}$, one point per step. Early pairs hand weight to points that are not support vectors; as the KKT conditions are enforced the weight drains into $(3,3)$ and $(1,1)$ and settles at exactly $\frac{1}{4}$ each.

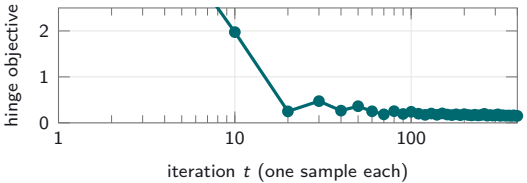
- Every step keeps $\sum_i \alpha_i y_i = 0$ and $\alpha_i \geq 0$: the iterate is always feasible.
- The dual objective never decreases: each pair step is an exact maximisation.

Insight

The same code, with K_{ij} read from a Gaussian Gram matrix, solved the rings. That is what "the kernel trick" buys an algorithm designer: one solver for every lift.

Interactive explorer: watch the multipliers move, one pair at a time ↗

Pegasos: Stochastic Subgradient On The Primal



Insight

400 single-sample steps bring the direction of $\mathbf{w}$ to 45.6° (exact: 45°); each costs $O(d)$. The hinge terms are *averaged*, so $\lambda = 1/(nC)$: here $\lambda = 0.1$, $n = 8$, $C = 1.25$.

A *subgradient*: at the hinge's kink, $m = 1$, any slope in $[-1, 0]$ is allowed.

The soft-margin problem, averaged over the samples:

$$\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_i \max\bigl(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\bigr),$$

Draw one sample i_t per step and step along a *subgradient*:

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\mathbf{w} + \eta_t\, y_{i_t} \mathbf{x}_{i_t}\, \mathbb{I}_t,$$
$$b \leftarrow b + \eta_t\, y_{i_t}\, \mathbb{I}_t,$$

with $\mathbb{I}_t = \mathbb{I}[y_{i_t} f(\mathbf{x}_{i_t}) < 1]$ and $\eta_t = 1/(\lambda t)$. Shrink $\mathbf{w}$ only (the penalty has no b), then, if the sample is inside the corridor, push toward it.

Warning

Primal SGD has no kernel: it needs $\mathbf{w}$ explicitly. It is the algorithm for a linear SVM on millions of samples.

An SVM Outputs A Score, Not A Probability

$f(\mathbf{x}) = \sum_{i \in \mathrm{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$ is a signed, distance-like score. It ranks; it does not estimate a probability.

- **Threshold** it at 0 for the maximum-margin decision, or elsewhere. Sweeping the threshold gives the ROC curve; AUC measures how well f orders positives above negatives, exactly as in the evaluation lecture.
- **Costs.** To use $\tau^* = c_{\mathrm{FP}}/(c_{\mathrm{FP}} + c_{\mathrm{FN}})$ you need a probability. Fit a one-dimensional logistic regression from f to y on validation data (Platt scaling), then threshold that.
- **Scale the features first.** Margins, distances and Gaussian kernels are in the units of $\mathbf{x}$; a feature in millimetres dominates one in metres.
- **Multi-class.** One machine per class against the rest, or per pair; take the largest score. The binary machine is the unit.

The Procedure

Training and using an SVM

1. Standardise features on training statistics; apply the same numbers everywhere.
2. Choose the kernel, C and (for the Gaussian kernel) γ by validation error over a grid.
3. Refit on training plus validation data with the chosen settings.
4. If a probability is needed, calibrate f on held-out data; choose the operating threshold from the costs.
5. Report the test-set confusion matrix, AUC and cost, once.

Warning

Steps 1, 2 and 5 are the same as for Ridge and for logistic regression. The SVM changes the model, not the discipline.

Why This Construction Is Worth Copying

The template.

1. Say what "best" means in words, then as an objective with constraints.
2. Check convexity; then the answer is unique and any local method finds it.
3. Write the Lagrangian; the multipliers tell you *which data matter*.
4. Look at where the data enter the solution; if only through inner products, you have a kernel method for free.
5. Only then choose an algorithm, by asking what it costs per step and in memory.

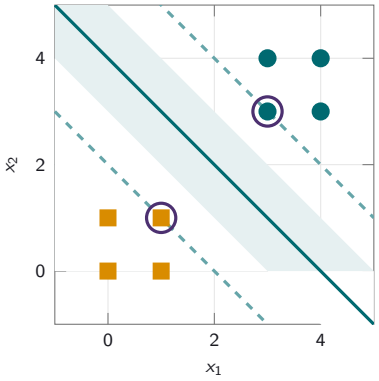
Where you will meet it again.

- Kernel ridge regression and Gaussian processes: the same trick on least squares.
- Boosting and neural networks: lifts that are learned rather than fixed.

Summary

An SVM is not a black box with a kernel knob. It is a worked answer to "which line?" whose every part follows from that question.

Summary



spam ($y = +1$) not spam ($y = -1$)

$\mathbf{w} = (\frac{1}{2}, \frac{1}{2})$, $b = -2$, margin 1.414, two support vectors with $\alpha = \frac{1}{4}$

1. **The rule.** Among separating lines, take the one farthest from its nearest point. Distance is score over $\|\mathbf{w}\|$; fix the scale so the nearest score is 1; then: minimise $\frac{1}{2}\|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$. Convex, unique.
2. **Dual.** $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$ with $\alpha_i > 0$ only on the corridor's edge: the support vectors. On our data, two of eight, $\alpha = \frac{1}{4}$.
3. **Kernel.** The dual and the decision function see the data only through inner products; replace them by $K(\mathbf{x}, \mathbf{z})$ and the line lives in a lifted space you never construct.
4. **Soft margin.** Slack at price C; equivalently hinge loss plus the Ridge penalty. C and γ by validation.
5. **Solvers.** SMO on the dual (two multipliers at a time, kernels, n^2 memory); stochastic subgradient on the primal (linear, any n).
6. **Use.** The output is a score: threshold, ROC, calibration and costs as in the evaluation lecture.

Check The Reasoning, Not Just The Formula

1. Two support vectors at distance D apart, one per class. What is α? What is the margin?
2. A point far from the boundary is removed from the training set. What happens to the SVM? To a logistic regression?
3. Why can the dual, but not the primal, be kernelised?
4. You double every feature's units. What happens to a linear SVM's decisions? To a Gaussian-kernel SVM with fixed γ?
5. As $C \to \infty$, what does the soft-margin machine become? As $C \to 0$?
6. Hinge loss is zero for a point with margin 1.5; logistic loss is not. What does that imply for which points influence each model?
7. Why does SMO update two multipliers rather than one?

Open the support vector machine explorer ↗